# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02**

**COURSE OUTCOME COVERED**

***CO3:*** *Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis. (BL4)*

*Assessment Tool Weightage: 50%*

**QUESTIONS: 5 Total Marks:50** Annexure A

## EXPERIMENT

***Code of Conduct:***

*I Peram.Mahitha(192524136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning | Marks |
|---|---|---|---|---|---|---|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts | |
| Experimental Design | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation | |
| Use of Tools | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools | |
| Analysis of Results | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results | |

| | | | | | | |
|---|---|---|---|---|---|---|
| Documentation | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation | |

Annexure A

# EXPERIMENT

**Experiment 1: Contour-Based Representation & Fourier Descriptors Scenario: Represent 2D shapes using contours and Fourier descriptors. Input Data:**

**Aim :**

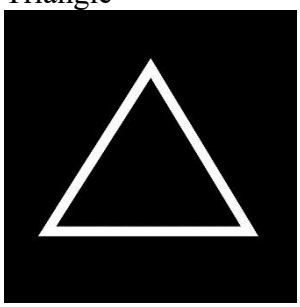
To represent 2D shapes using contours and Fourier descriptors and analyze the shape representation.

**A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star).**

**Answer :**

**Shape Images**

- Binary images of 2D hand-drawn letters A–Z or simple symbols such as:
  - Triangle



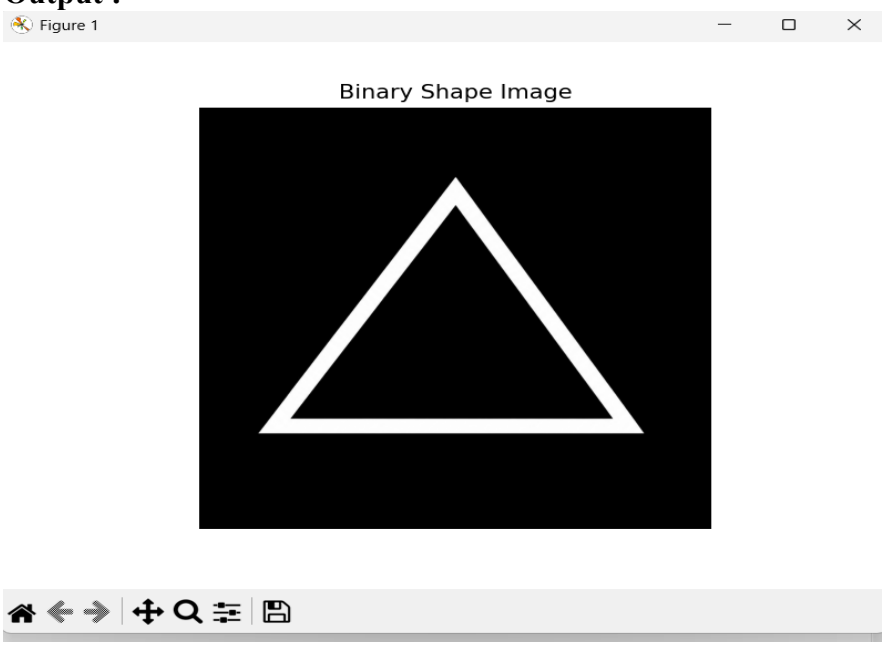
- The image should contain a black/white shape with a clear boundary.

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import os
# Image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\triangle.png"
# Check whether image exists
print("File exists:", os.path.exists(image_path))
# Load image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
# Check whether image was loaded
if image is None:
```

```
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully")
    print("Image size:", image.shape)
    # Display image
    plt.imshow(image, cmap='gray')
    plt.title("Binary Shape Image")
    plt.axis("off")
    plt.show()
    # Find contours
    contours, _ = cv2.findContours(
        image,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_NONE
    )
    # Draw contours
    output = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
    cv2.drawContours(
        output,
        contours,
        -1,
        (0, 255, 0),
        2
    )
    plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
    plt.title("Detected Contour")
    plt.axis("off")
    plt.show()
    print("Number of contours:", len(contours))
```

**Output :**



**B. Image size: 256 × 256 pixels**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
# Image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\triangle.png"
# Load image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    # Resize image to 256 × 256
    image_256 = cv2.resize(image, (256, 256))
    # Display image
    plt.imshow(image_256, cmap='gray')
    plt.title("Image Size: 256 × 256 Pixels")
    plt.axis("off")
    plt.show()
    # Display image dimensions
    height, width = image_256.shape
    print("Height:", height)
    print("Width:", width)
    print("Image Size:", width, "×", height)
```

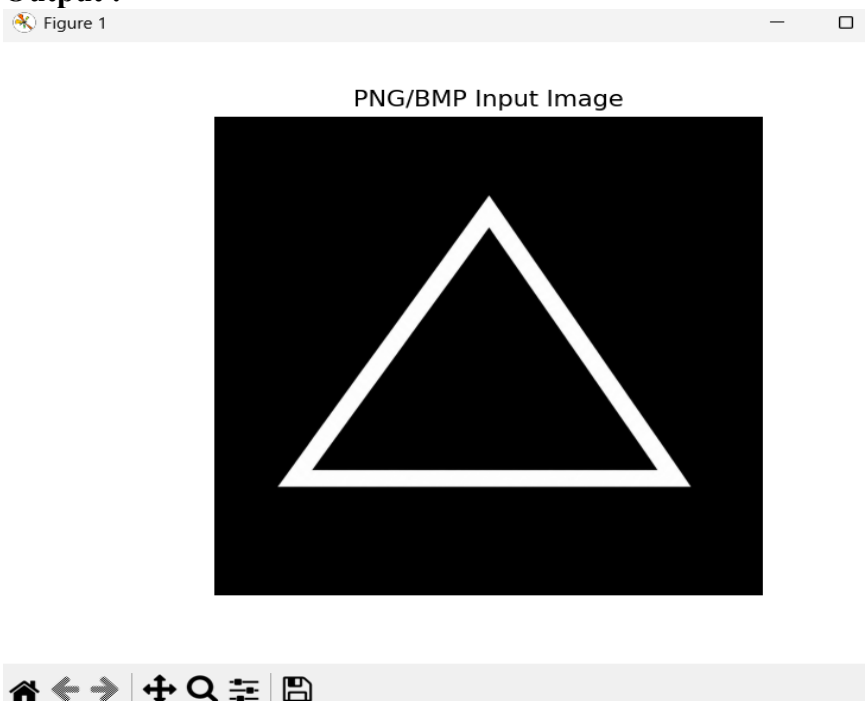
**Output :**

C. **C. File format: PNG or BMP**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import os
# Image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\triangle.png"
# Check file extension
file_extension = os.path.splitext(image_path)[1].lower()
print("File name:", os.path.basename(image_path))
print("File format:", file_extension)
# Check whether the format is PNG or BMP
if file_extension == ".png" or file_extension == ".bmp":
    print("Valid file format: PNG or BMP")
else:
    print("Invalid file format")
# Load image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully")
    # Display image
    plt.imshow(image, cmap="gray")
    plt.title("PNG/BMP Input Image")
    plt.axis("off")
    plt.show()
```

**Output :**



**Experiment 2: Region-Based Representation & Medial Axis Scenario: Analyze object shapes in binary images using medial axis. Input Data:**

**Aim**

To analyze the shape of a binary industrial component using medial axis representation and identify defects such as holes, protrusions, and missing parts.

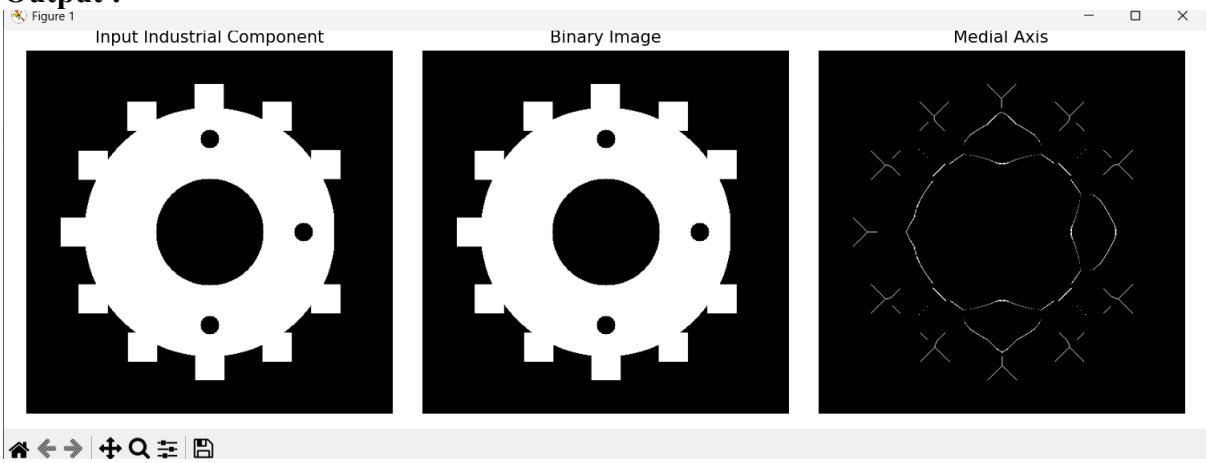
**A. Binary images of industrial components (e.g., gears, machine parts)**

**Code :**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Load the input image
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\gear_defective.png"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
    exit()
print("Image loaded successfully")
print("Original image size:", image.shape)
# 2. Resize image to 512 × 512
image = cv2.resize(image, (512, 512))
print("Resized image size:", image.shape)
# 3. Convert image into binary image
_, binary = cv2.threshold(
    image,
    127,
    255,
    cv2.THRESH_BINARY
)
# 4. Create skeleton / medial-axis approximation
#    using morphological skeletonization
skeleton = np.zeros(binary.shape, np.uint8)
element = cv2.getStructuringElement(
    cv2.MORPH_CROSS,
    (3, 3)
)
temp = np.zeros(binary.shape, np.uint8)
eroded = np.zeros(binary.shape, np.uint8)
current = binary.copy()
while True:

    # Erode image
    eroded = cv2.erode(current, element)
    # Dilate eroded image
```

```
    opened = cv2.dilate(eroded, element)
    # Find difference
    temp = cv2.subtract(current, opened)
    # Add skeleton pixels
    skeleton = cv2.bitwise_or(skeleton, temp)
    # Continue erosion
    current = eroded.copy()
    # Stop when image becomes empty
    if cv2.countNonZero(current) == 0:
        break
# 5. Display results
plt.figure(figsize=(12, 4))
# Original image
plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Input Industrial Component")
plt.axis("off")
# Binary image
plt.subplot(1, 3, 2)
plt.imshow(binary, cmap="gray")
plt.title("Binary Image")
plt.axis("off")
# Medial axis / skeleton
plt.subplot(1, 3, 3)
plt.imshow(skeleton, cmap="gray")
plt.title("Medial Axis")
plt.axis("off")
plt.tight_layout()
plt.show()
# 6. Display information
print("Number of skeleton pixels:",
      cv2.countNonZero(skeleton))
print("Medial axis extraction completed successfully.")
```

**Output :**

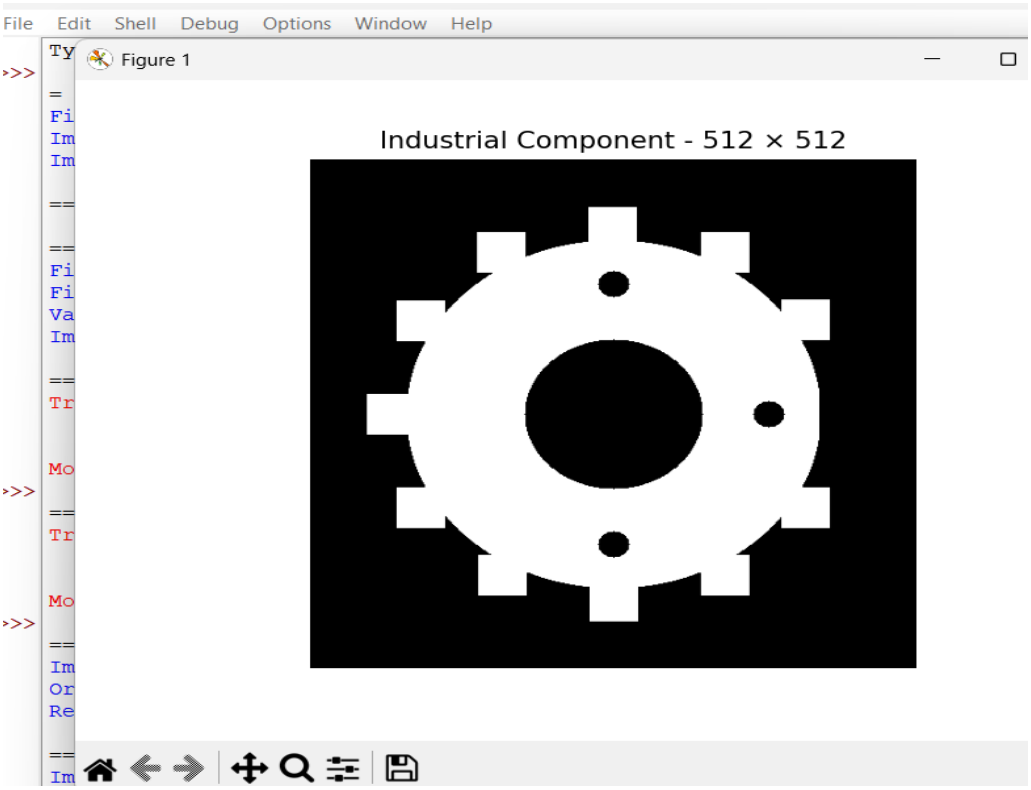


**B. Image size: 512 × 512 pixels**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
# Image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\gear_defective.png"
# Load image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Original image size:", image.shape)
    # Resize image to 512 × 512
    image_512 = cv2.resize(image, (512, 512))
    # Get image dimensions
    height, width = image_512.shape
    # Display image
    plt.imshow(image_512, cmap="gray")
    plt.title("Industrial Component - 512 × 512")
    plt.axis("off")
    plt.show()
    # Display size
    print("Height:", height)
    print("Width:", width)
    print("Image Size:", width, "×", height)
```

**Output :**



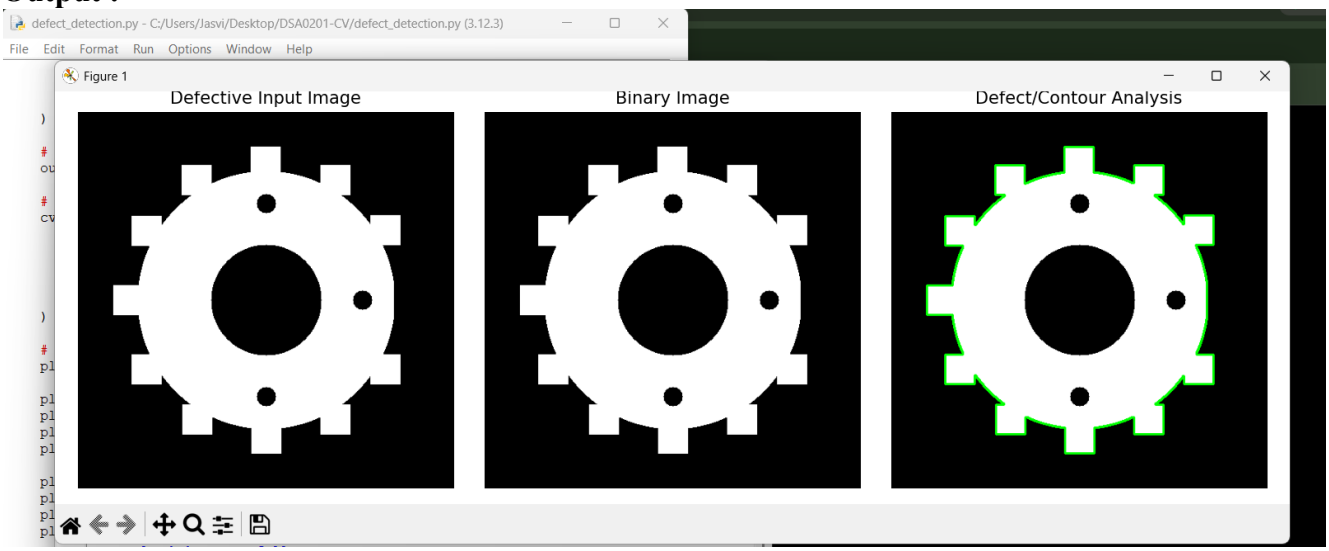
**C. Defects like small holes, protrusions, or missing parts included**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
# Input image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\gear_defective.png"
# Load image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    # Convert to binary image
    _, binary = cv2.threshold(
        image,
        127,
        255,
        cv2.THRESH_BINARY
    )
    # Find contours
    contours, _ = cv2.findContours(
        binary,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )
    # Create output image
    output = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
    # Draw detected contours
    cv2.drawContours(
        output,
        contours,
        -1,
        (0, 255, 0),
        2
    )
    # Display results
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(image, cmap="gray")
    plt.title("Defective Input Image")
    plt.axis("off")
    plt.subplot(1, 3, 2)
    plt.imshow(binary, cmap="gray")
    plt.title("Binary Image")
    plt.axis("off")
    plt.subplot(1, 3, 3)
    plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
```

```
plt.title("Defect/Contour Analysis")
plt.axis("off")
plt.tight_layout()
plt.show()
print("Number of detected regions:", len(contours))
print("Defect analysis completed.")
```

**Output :**



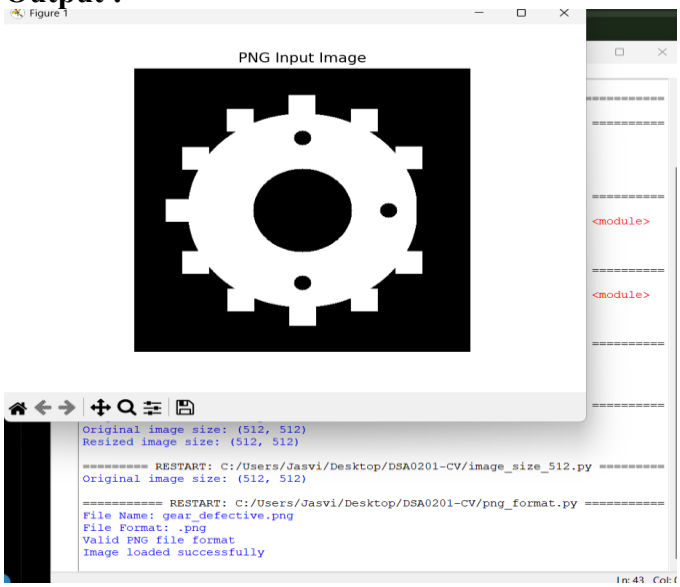

**D. File format: PNG**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import os
# Image path
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\gear_defective.png"
# Get file name and extension
file_name = os.path.basename(image_path)
file_format = os.path.splitext(image_path)[1].lower()
# Display file information
print("File Name:", file_name)
print("File Format:", file_format)
# Check whether the file is PNG
if file_format == ".png":
    print("Valid PNG file format")
else:
    print("Invalid file format. Please use PNG.")
# Load image
```

```
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully")
    # Display PNG image
    plt.imshow(image, cmap="gray")
    plt.title("PNG Input Image")
    plt.axis("off")
    plt.show()
```

**Output :**




**Experiment 3: Deformable Curves and Snakes (Active Contours) Scenario: Segment organs or objects with irregular boundaries. Input Data:**

**Aim**

To segment an object with an irregular boundary from a grayscale medical image using deformable curves (active contours/snakes).

**A. Grayscale medical images (e.g., liver or heart slices)**

**Code :**

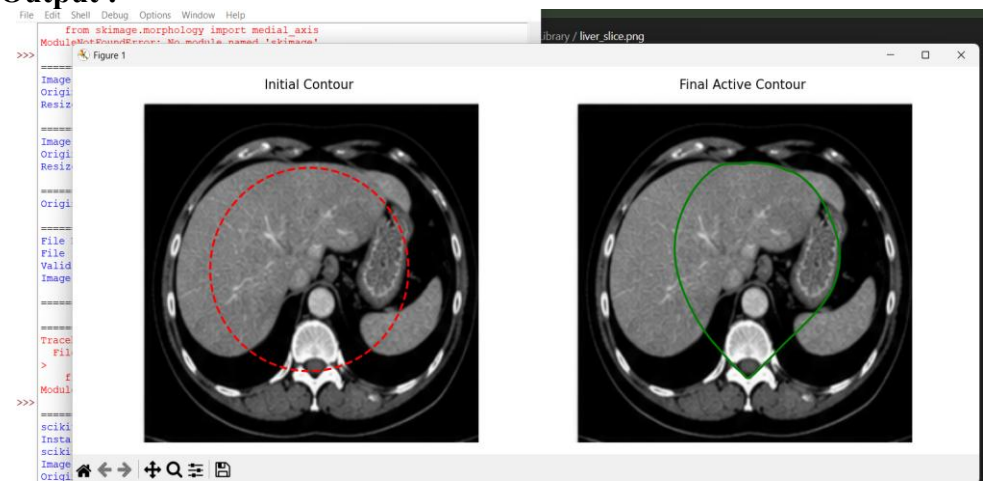
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import sys
import subprocess
# Install scikit-image automatically if not installed
try:
    from skimage.segmentation import active_contour
    from skimage.filters import gaussian
```

```
except ModuleNotFoundError:
    print("scikit-image is not installed.")
    print("Installing scikit-image...")
    subprocess.check_call([
        sys.executable,
        "-m",
        "pip",
        "install",
        "scikit-image"
    ])
    from skimage.segmentation import active_contour
    from skimage.filters import gaussian
print("scikit-image is ready.")
# 1. Load grayscale medical image
image_path = r"C:\Users\Jasvi\Desktop\DSA0201-CV\liver_slice.png"

image = cv2.imread(
    image_path,
    cv2.IMREAD_GRAYSCALE
)
if image is None:
    print("Error: Image could not be loaded.")
    exit()
print("Image loaded successfully.")
print("Original image size:", image.shape)
# 2. Resize image to 512 × 512
image = cv2.resize(
    image,
    (512, 512)
)
print("Image size after resizing:", image.shape)
# 3. Normalize image
image_float = image.astype(np.float64) / 255.0
# 4. Smooth image
smooth_image = gaussian(
    image_float,
    sigma=2
)
# 5. Create initial circular contour
height, width = image.shape
center_x = width / 2
center_y = height / 2
radius = 150
theta = np.linspace(
    0,
    2 * np.pi,
    400
)
```

```
x = center_x + radius * np.cos(theta)
y = center_y + radius * np.sin(theta)

initial_contour = np.array(
    [y, x]
).T
# 6. Apply Active Contour / Snake
print("Applying active contour...")
snake = active_contour(
    smooth_image,
    initial_contour,
    alpha=0.01,
    beta=10,
    gamma=0.001,
    max_num_iter=1000
)
# 7. Display input image
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(
    image,
    cmap="gray"
)
plt.plot(
    initial_contour[:, 1],
    initial_contour[:, 0],
    "--r",
    linewidth=2
)
plt.title("Initial Contour")
plt.axis("off")
# 8. Display final snake
plt.subplot(1, 2, 2)
plt.imshow(
    image,
    cmap="gray"
)
plt.plot(
    snake[:, 1],
    snake[:, 0],
    "-g",
    linewidth=2
)
plt.title("Final Active Contour")
plt.axis("off")
plt.tight_layout()
plt.show()
# 9. Completion message
```

```
print("Active contour segmentation completed successfully.")
print("Number of contour points:", len(snake))
```

**Output :**

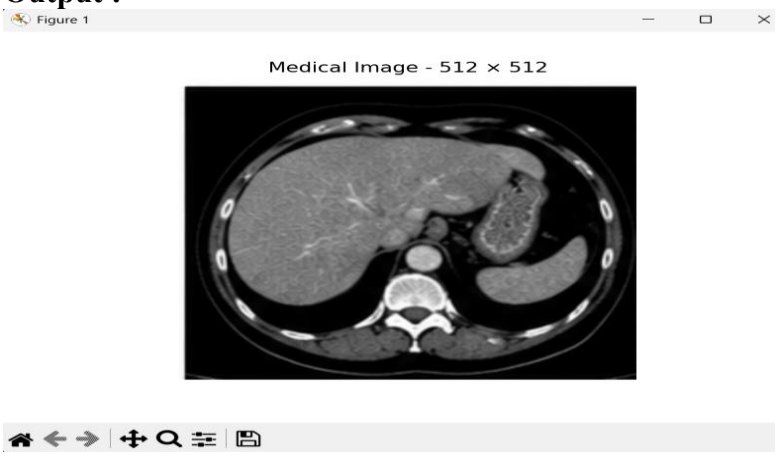



**B. Image size: 512 × 512 pixels**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\liver_slice.png"
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    image_512=cv2.resize(image,(512,512))
    height,width=image_512.shape
    plt.imshow(image_512,cmap="gray")
    plt.title("Medical Image - 512 × 512")
    plt.axis("off")
    plt.show()
    print("Height:",height)
    print("Width:",width)
    print("Image Size:",width,"×",height)
```

**Output :**

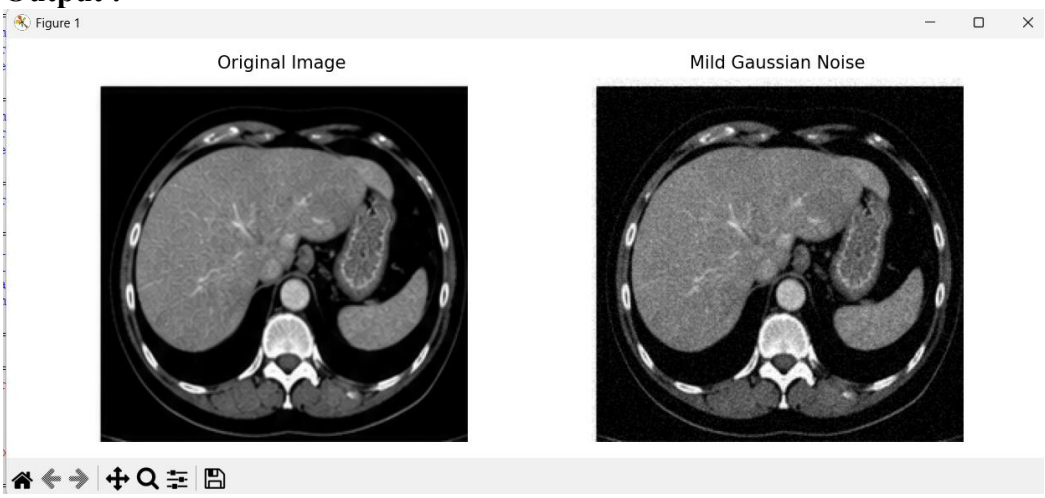


**C. Noise: mild Gaussian noise**

**Code :**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\liver_slice.png"
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    image_float=image.astype(np.float32)
    noise=np.random.normal(0,10,image.shape)
    noisy_image=image_float+noise
    noisy_image=np.clip(noisy_image,0,255).astype(np.uint8)
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.imshow(image,cmap="gray")
    plt.title("Original Image")
    plt.axis("off")
    plt.subplot(1,2,2)
    plt.imshow(noisy_image,cmap="gray")
    plt.title("Mild Gaussian Noise")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
    print("Mild Gaussian noise added successfully.")
```

**Output :**



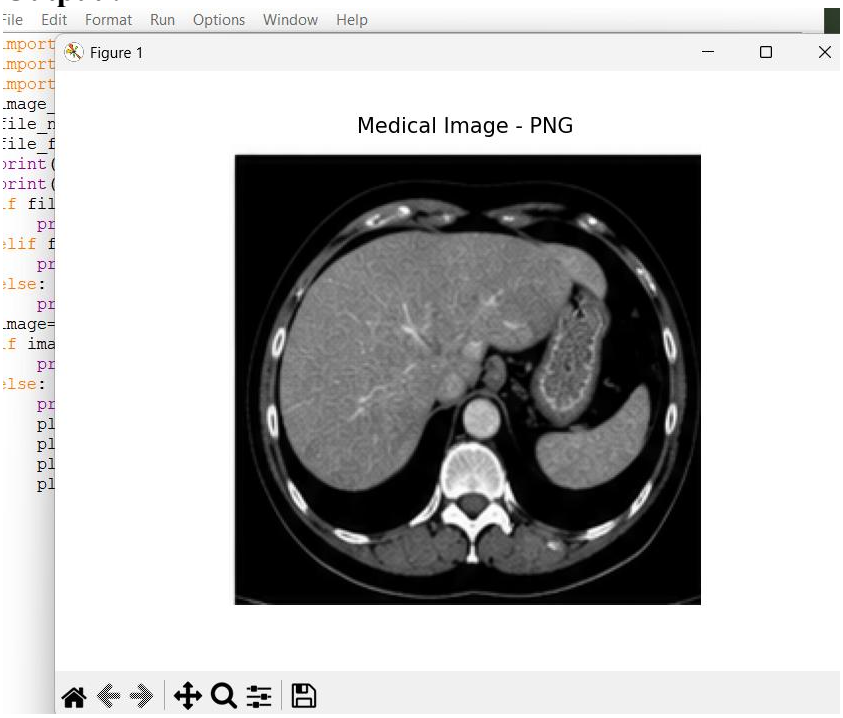

**D. File format: DICOM or PNG**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import os
```

```
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\liver_slice.png"
file_name=os.path.basename(image_path)
file_format=os.path.splitext(image_path)[1].lower()
print("File Name:",file_name)
print("File Format:",file_format)
if file_format==".png":
    print("Valid file format: PNG")
elif file_format==".dcm":
    print("Valid file format: DICOM")
else:
    print("Invalid file format")
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Medical image loaded successfully.")
    plt.imshow(image,cmap="gray")
    plt.title("Medical Image - PNG")
    plt.axis("off")
    plt.show()
```

**Output :**

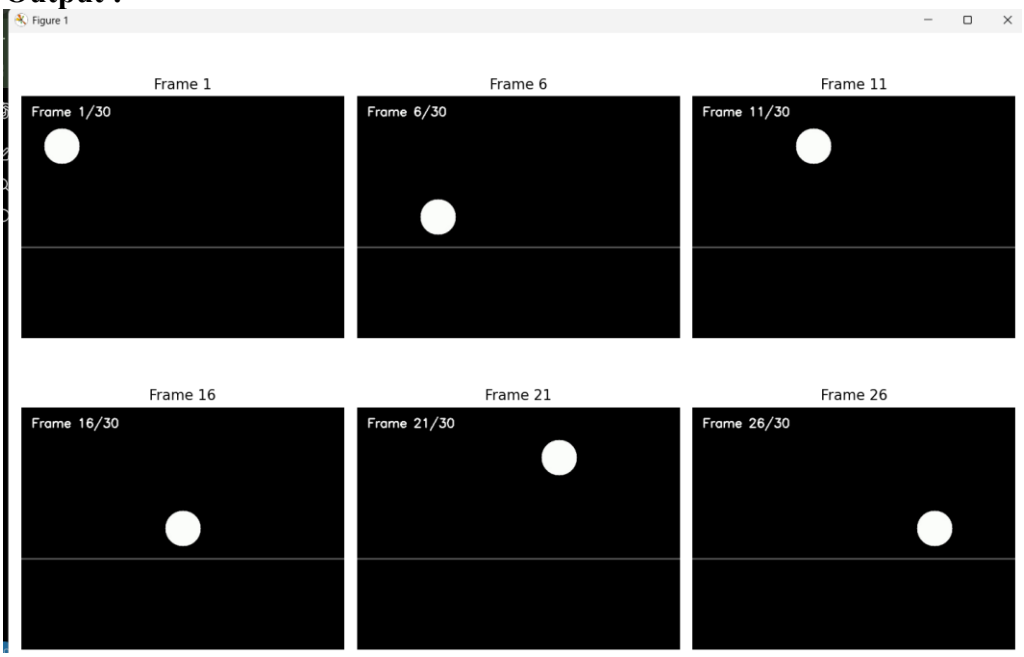



**Experiment 4: Level Set Methods**
**Scenario: Track moving objects with evolving boundaries.**
**Input Data:**

**A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or moving hand)**
**Code :**

```
import cv2
import matplotlib.pyplot as plt
video_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\bouncing_ball_30frames.mp4"
cap=cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: Video could not be opened.")
else:
    total_frames=int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    print("Total Frames:",total_frames)
    frames=[]
    for i in range(30):
        ret,frame=cap.read()
        if not ret:
            break
        frame=cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)
        frames.append(frame)
    cap.release()
    print("Frames Read:",len(frames))
    plt.figure(figsize=(12,8))
    for i in range(6):
        plt.subplot(2,3,i+1)
        plt.imshow(frames[i*5])
        plt.title("Frame "+str(i*5+1))
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

**Output :**

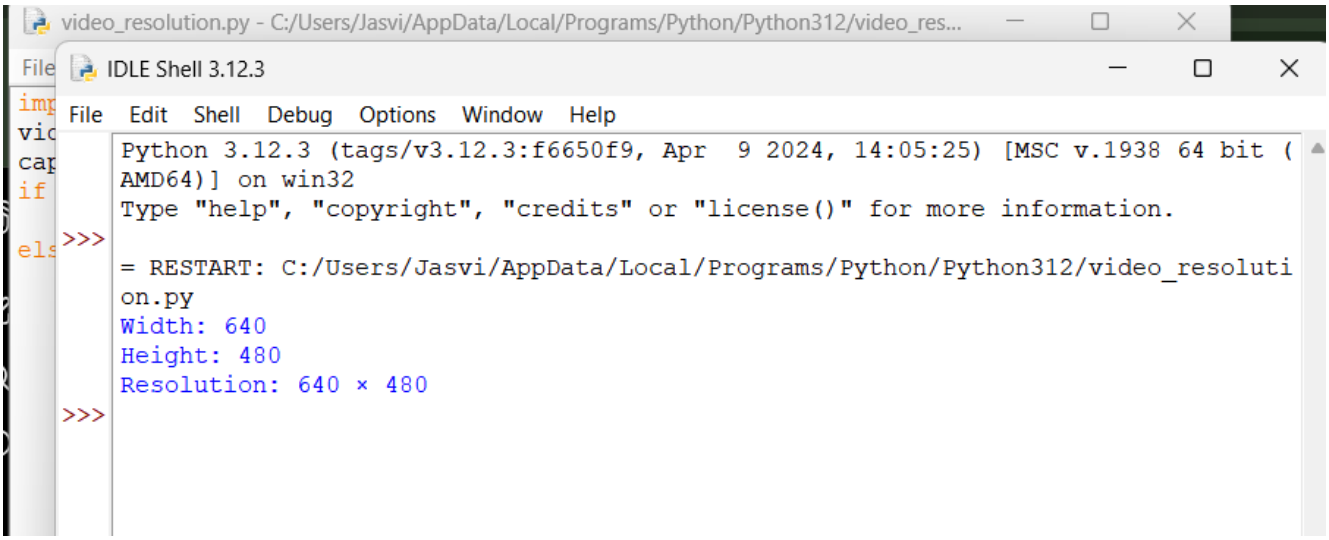



**B. Resolution: 640 × 480 pixels**

**Code :**

```
import cv2
video_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\bouncing_ball_30frames.mp4"
cap=cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: Video could not be opened.")
else:
    width=int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height=int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    print("Width:",width)
    print("Height:",height)
    print("Resolution:",width,"×",height)
    cap.release()
```

**Output :**



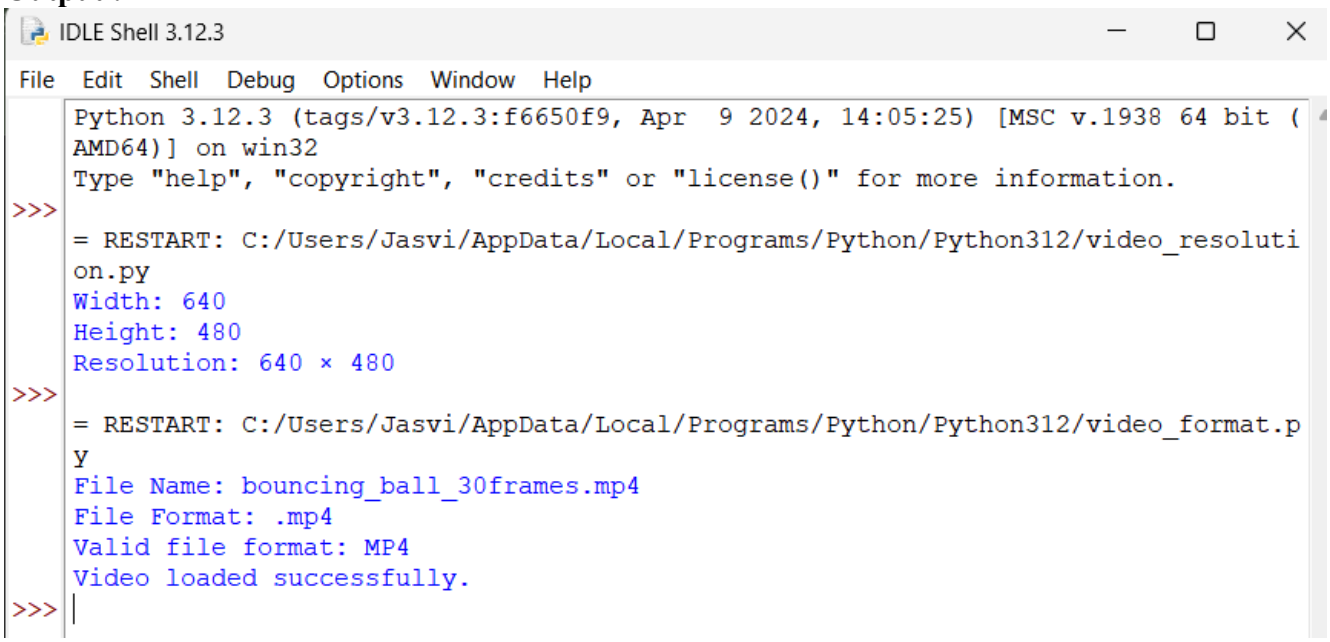

**C. Format: MP4 or sequence of PNG images**

**Code :**

```
import cv2
import os
video_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\bouncing_ball_30frames.mp4"
file_name=os.path.basename(video_path)
file_format=os.path.splitext(video_path)[1].lower()
print("File Name:",file_name)
print("File Format:",file_format)
if file_format==".mp4":
    print("Valid file format: MP4")
elif file_format==".png":
    print("Valid file format: PNG")
else:
    print("Invalid file format")
cap=cv2.VideoCapture(video_path)
if cap.isOpened():
    print("Video loaded successfully.")
```

```
    cap.release()
else:
    print("Error: Video could not be loaded.")
```

**Output :**




**Experiment 5: Multi-Resolution & Wavelet Descriptors**
**Scenario: Shape recognition at multiple scales.**
**Input Data:**

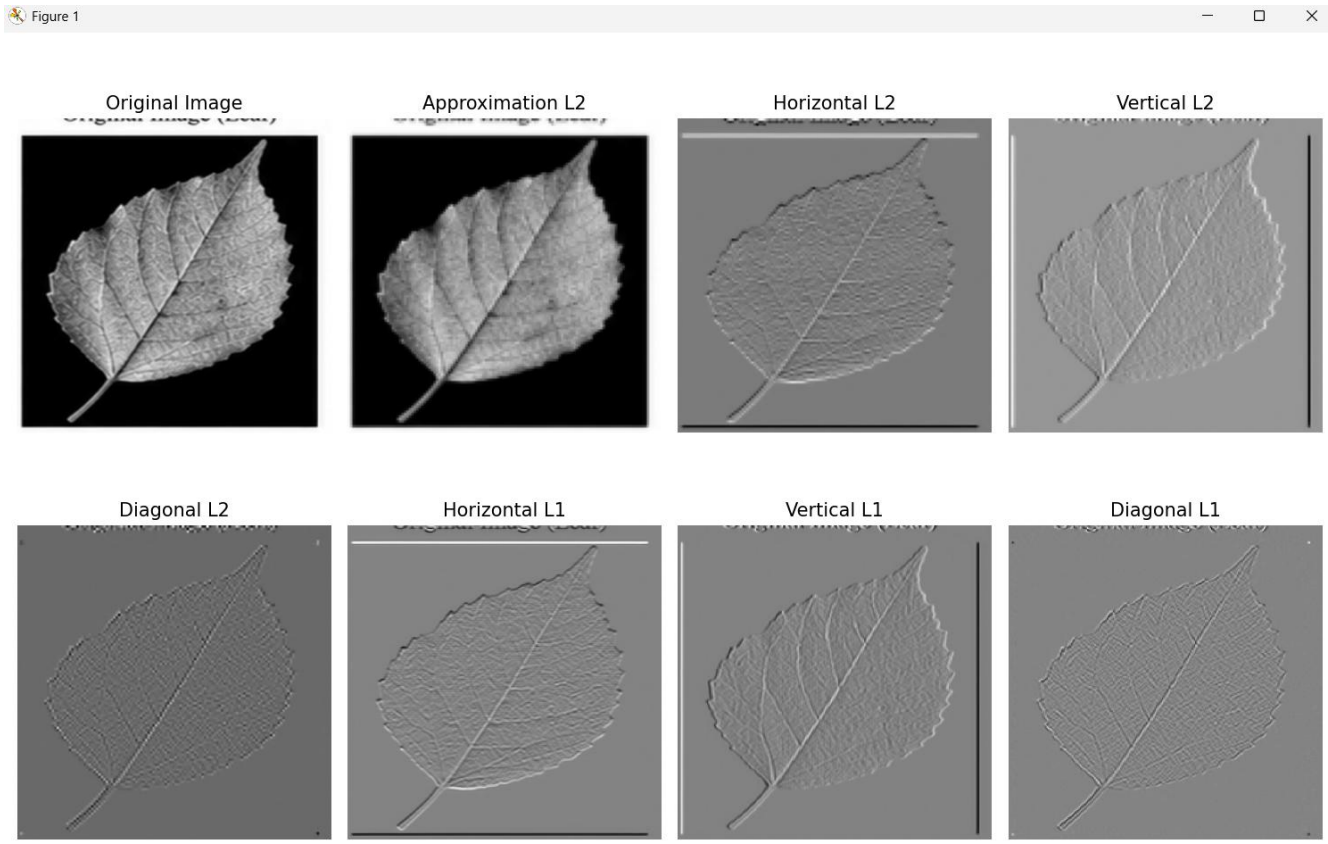
**A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)**

**Code :**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import sys
import subprocess
try:
    import pywt
except ModuleNotFoundError:
    print("Installing PyWavelets...")
    subprocess.check_call([sys.executable,"-m","pip","install","PyWavelets"])
    import pywt
print("PyWavelets is ready.")
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\leaf_original.png"
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
    exit()
print("Image loaded successfully.")
print("Image size:",image.shape)
image=cv2.resize(image,(512,512))
```

```
coeffs=pywt.wavedec2(image,"haar",level=2)
cA2,(cH2,cV2,cD2),(cH1,cV1,cD1)=coeffs
images=[image,cA2,cH2,cV2,cD2,cH1,cV1,cD1]
titles=["Original Image","Approximation L2","Horizontal L2","Vertical L2","Diagonal
L2","Horizontal L1","Vertical L1","Diagonal L1"]
plt.figure(figsize=(12,8))
for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(images[i],cmap="gray")
    plt.title(titles[i])
    plt.axis("off")
plt.tight_layout()
plt.show()
print("Wavelet decomposition completed successfully.")
print("Wavelet: Haar")
print("Decomposition Level: 2")
```

**Output :**



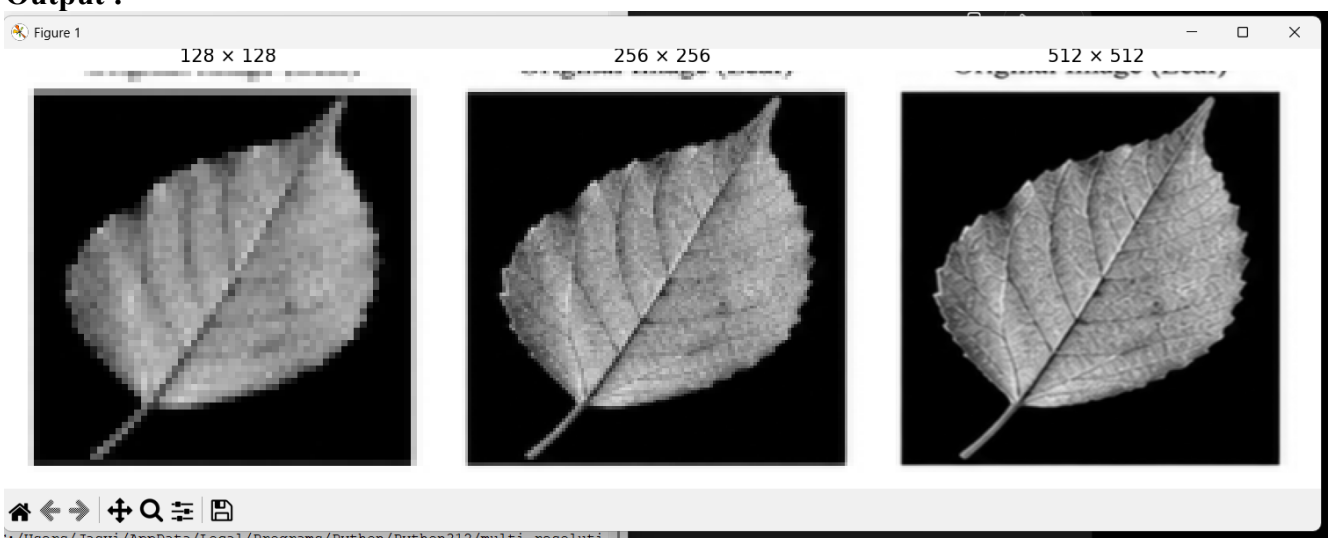

**B. Image sizes: 128 × 128, 256 × 256, 512 × 512 pixels**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import sys
import subprocess
```

```
try:
    import pywt
except ModuleNotFoundError:
    print("Installing PyWavelets...")
    subprocess.check_call([sys.executable,"-m","pip","install","PyWavelets"])
    import pywt
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\leaf_original.png"
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
    exit()
sizes=[128,256,512]
plt.figure(figsize=(12,4))
for i,size in enumerate(sizes):
    resized=cv2.resize(image,(size,size))
    cA,(cH,cV,cD)=pywt.dwt2(resized,"haar")
    plt.subplot(1,3,i+1)
    plt.imshow(cA,cmap="gray")
    plt.title(str(size)+" × "+str(size))
    plt.axis("off")
    print("Image Size:",size,"×",size)
plt.tight_layout()
plt.show()
print("Multi-resolution wavelet analysis completed.")
```

**Output :**




**C. Format: PNG**

**Code :**

```
import cv2
import matplotlib.pyplot as plt
import os
image_path=r"C:\Users\Jasvi\Desktop\DSA0201-CV\leaf_original.png"
file_name=os.path.basename(image_path)
file_format=os.path.splitext(image_path)[1].lower()
```

```
print("File Name:",file_name)
print("File Format:",file_format)
if file_format==".png":
    print("Valid file format: PNG")S
else:
    print("Invalid file format")
image=cv2.imread(image_path,cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully.")
    plt.imshow(image,cmap="gray")
    plt.title("PNG Grayscale Image")
    plt.axis("off")
    plt.show()
```

**Output :**